



2024 - 25

CS -22

Programming
with
JAVA



Bhavesh K. Hadiyal

HOD of Computer science

B.C.A. Department

OMVVIM College

Unit – 1
History,
Introduction
And
Language Basics,
Classes and Objects

Index

- History and Features of Java
- Java Editions
- JDK, JVM and JRE
- JDK Tools
- Compiling and executing basic Java Program
- Java IDE (Net Beans and Eclipse)
- Data Type (Integer, Float, Character, Boolean)
- Java Tokens (Keyword, Literal, Identifier, Whitespace, Separators, Comments, Operators)
- Operators (Arithmetic, Relational, Boolean Logical, Bitwise Logical, Assignment, Unary, Shift, Special operators)
- Java Keywords (assert, strictfp, enum)
- Type Casting
- Decision Statements (if, switch)
- Looping Statements (for, while, do...while)
- Jumping Statements (break, continue, and return)
- Array (One Dim., Rectangular, Jagged)
- Command Line Argument Array
- OOP Concepts (Class, Object, Encapsulation, Inheritance, Polymorphism)
- Creating and using Class with members
- Constructor
- finalize () method
- Static and Non-Static Members
- Overloading (Constructor & Method)
- Varargs, IIB (Instance Initialization Block) in Java

History of Java

- Java is one of the most popular programming languages used by software developers all over the world.
- Java was developed in the early 1990s by James Gosling as an object-oriented programming language.
- The history of Java starts with the **Green Team**. **James Gosling**, **Mike Sheridan**, and **Patrick Naughton** initiated the Java language project in June 1991.
- The small team of sun engineers called Green Team.
- Initially it was designed for small, embedded systems in electronic appliances like set-top boxes, televisions, etc.
- Originally C++ was considered to be used in the project but the idea was rejected for several reasons (For instance C++ required more memory).
- Firstly, it was called “**Greentalk**” by **James Gosling**, and the file extension was **.gt**. After that, it was called Oak and was developed as a part of the Green project.
- Oak is a symbol of strength and chosen as a national tree of many countries like the U.S.A., France, Germany, Romania, etc. In 1995, Oak was renamed as “Java” because it was already a trademark by Oak Technologies.
- The team gathered to choose a new name. The suggested words were “dynamic”, “revolutionary”, “Silk”, “jolt”, “DNA”, etc. They wanted something that reflected the essence of the technology: revolutionary, dynamic, lively, cool, unique, and easy to spell, and fun to say. According to **James Gosling**, “Java was one of the top choices along with Silk”. Since Java was so unique, most of the team members preferred Java than other names.
- **Java is an island in Indonesia** where the first coffee was produced (called Java coffee).
- Notice that Java is just a name, **not an acronym**. In 1995, Time magazine called Java one of the Ten Best Products of 1995.
- Java was created on the principles like Robust, Portable, Platform Independent, High Performance, Multithread, etc. and was called one of the Ten Best Products of 1995 by the TIME MAGAZINE.
- Initially developed by James Gosling at **Sun Microsystems** (which is now a subsidiary of Oracle Corporation) and released in 1995.
- Sun officially presented the Java language at SunWorld on May 23, 1995. Then, in 2009, the Oracle company bought the Sun company, which explains why the language now belongs to Oracle
- JDK 1.0 was released on January 23, 1996. After the first release of Java, there have been many additional features added to the language. Now Java is being used in Windows applications, Web applications, enterprise applications, mobile applications, cards, etc. Each new version adds new features in Java.
- Currently, Java is used in internet programming, mobile devices, games, e-business solutions, etc.

Features of Java

A list of the most important features of the Java language is given below.

Simple -> Java is very easy to learn, and its syntax is simple, clean and easy to understand.

Object-Oriented -> In Java, everything is an Object. Java can be easily extended since it is based on the Object model.

Portable -> Java is portable because it facilitates you to carry the Java bytecode to any platform.

Platform independent -> you can run Java bytecode on any hardware. In Java, the size of the primitive data types is machine-independent

Secured -> Java supports access modifiers to check memory access, Programs run in a virtual machine sandbox and it has a bytecode verifier

Robust -> Java is robust as it is capable of handling run-time errors, supports automatic garbage collection and exception handling

Architecture neutral -> Java compiler generates an architecture-neutral object file format.

Interpreted -> Java byte code is translated on the fly to native machine instructions and is not stored anywhere.

High Performance -> with the use of **JIT – Just In Time compiler**, Java enables high performance.

Multithreaded -> Multithreaded means handling multiple tasks simultaneously or executing multiple portions (functions) of the same program in parallel.

Distributed -> **RMI (Remote Method Invocation)** and **EJB (Enterprise Java Bean)** are used for creating distributed applications. This feature of Java makes us able to access files by calling the methods from any machine on the internet.

Dynamic -> it supports the dynamic loading of classes. It means classes are loaded on demand.

Java Editions

1. Java platform, Standard Edition (Java SE)
2. Java platform, Enterprise Edition (Java EE)
3. Java platform, Micro Edition (Java ME)
4. JavaFX

1. Java platform, Standard Edition (Java SE):-

It is made up of **APIs (Application Programming Interfaces)** that provide Java's fundamental programming features.

It is the minimum requirement to run a Java application.

It uses to develop stand-alone applications that run without interacting with other devices over the Internet or on any other medium.

Some of its components:

Java Virtual Machine (JVM), Java Class Library (JCL), Java Runtime Environment (JRE)

Java Development Kit (JDK)

[**GO to index page**](#)

2. Java platform, Enterprise Edition (Java EE):-

It is based on Java SE specification. It is mainly used for developing web & enterprise applications. The most recent version of this edition includes **more than 40 specifications** that assist programmers in building applications that use web services to exchange data, transform object-oriented data into entity-relationship models, manage transactional interactions, and other things.

The most used features of Java EE are:

Java Server Pages (JSP): It is a server-side programming language.

Java Server Faces (JSF): The JSF API offers elements such as input Text, command Buttons, etc.

Java Persistence API (JPA): Developers get access to an object/relational mapping mechanism.

Enterprise Java Beans (EJB): It is used for the development of corporate applications.

3. Java platform, Micro Edition (Java ME):-

It uses the Java programming language and provides APIs to develop **Mobile applications**.

This API is a subset of the Java SE API, along with special class libraries useful for small device application development.

Applications which are developed on Java ME are portable and can run across various devices. It provides functionalities such as networking, communication with native operating systems of mobile devices.

4. JavaFX :-

JavaFX stands for special effects in the Java language. It is used to develop Desktop applications as well as Rich Internet Applications. It uses a lightweight user-interfaces API.

These applications can run on multiple platforms including Web, Mobile and Desktops. It is intended to replace swing in Java applications as a GUI framework.

JavaFX applications use hardware-accelerated graphics and media engines to take advantage of higher-performance clients and a modern look-and-feel.

JDK, JVM and JRE

1. JDK (Java Development Kit)

JDK is the core component of Java Environment and provides all the tools, executables, and binaries required to compile, debug, and execute a Java Program.

JDK is a **platform-specific software** and that's why we have separate installers for Windows, Mac, and Unix systems.

We can say that JDK is the superset of JRE since it contains **Java Run Environment (JRE)** with Java compiler, debugger, and core classes.

JDK contains: **Java Runtime Environment (JRE)**, **An interpreter/loader (Java)**, **A compiler (javac)**, **An archiver (jar)** and many more.

2. JVM (Java virtual machine)

JVM is the heart of Java programming language. When we execute a Java program, JVM is responsible for converting the byte code to the machine-specific code.

JVM is platform-dependent and provides core java functions such as memory management, garbage collection, security, etc. JVM is customizable and we can use java options to customize it. For example, allocating minimum and maximum memory to JVM. [**GO to index page**](#)

JVM is called virtual because it provides an interface that does not depend on the underlying operating system and machine hardware. This makes java program write-once-run-anywhere. **JVM (Java Virtual Machine)** is an abstract machine. It is a specification that provides runtime environment in which java bytecode can be executed.

JVMs are available for many hardware and software platforms. JVM, JRE and JDK are platform dependent because configuration of each OS differs. But, Java is platform independent.

The JVM performs following main tasks:

Loads code, Verifies code, Executes code, Provides runtime environment

3. JRE (Java Run Environment)

JRE is the implementation of JVM. It provides a platform to execute java programs. JRE consists of JVM, Java binaries, and other classes to execute any program successfully.

JRE doesn't contain any development tools such as **Java compiler, debugger, JShell**, etc. If you just want to execute a java program, you can install only JRE. You don't need JDK because there is no development or compilation of java source code is required.

All the versions of JDK come bundled up with the JRE. This way, a user doesn't have to download and install JRE on their PC separately. It consists of **a library set + a few more files** that the **JVM (Java Virtual Machine)** deploys at the runtime.

It is used to provide runtime environment. It is the implementation of JVM. It physically exists.

JDK Tools

Standard JDK Tools and Utilities

Basic Tools – (javac, java, javadoc, apt, appletviewer, jar, jdb, javah, javap, extcheck)

Security Tools – (keytool, jarsigner, policytool, kinit, klist, ktab)

Internationalization Tools – (native2ascii)

Remote Method Invocation (RMI) Tools – (rmic, rmiregistry, rmid, serialver)

Java IDL and RMI-IIOP Tools – (tnameserv, idlj, orbd, servertool)

Java Deployment Tools – (pack200, unpack200)

Java Plug-in Tools – (htmlconverter)

Java Web Start Tools – (javaws)

Basic Tools: These tools are the foundation of the JDK. They are the tools you use to create and build applications.

Tool Name	Brief Description
javac	The compiler for the Java programming language.
java	The launcher for Java applications. In this release, a single launcher is used both for development and deployment. The old deployment launcher, jre, is no longer provided.
javadoc	API documentation generator.
apt	Annotation processing tool.
appletviewer	Run and debug applets without a web browser.
jar	Create and manage Java Archive (JAR) files.
jdb	The Java Debugger. See JPDA for the debugger architecture specifications.
javah	C header and stub generator. Used to write native methods.
javap	Class file disassembler
extcheck	Utility to detect Jar conflicts.

[GO to index page](#)

Compiling and Executing basic Java Program

Let's look at how to save the file, compile, and run the program. Please follow the subsequent steps –

- Open notepad or any IDE and write the code of your program.
- Save the file as: MyFirstJavaProgram.java.
- Open a command prompt window and go to the directory where you saved the class. Assume it's C:\.
- Type 'javac MyFirstJavaProgram.java' and press enter to compile your code. If there are no errors in your code, the command prompt will take you to the next line (Assumption: The path variable is set).
- Now, type ' java MyFirstJavaProgram ' to run your program.
- You will be able to see output on the window.

Java IDE (NetBeans and Eclipse)

A Java IDE is an integrated development environment for programming in Java; many also provide functionality for other languages,

IDEs typically provide a code editor, a compiler or interpreter and a debugger that the developer accesses through a unified graphical user interface (GUI). Java IDEs also include language-specific elements such as Ant and Maven build tools and TestNG and JUnit testing.

NetBeans

A Java-based IDE and underlying application platform framework. In addition to Java, JavaScript and JavaFX, NetBeans supports C/C++, PHP, Groovy, and HTML5.

Pro: Beginner-Friendly: NetBeans can be more beginner-friendly than Eclipse. It has a simpler user interface. And you can get started without all the plugins.

Pro: Stability: NetBeans is more stable. This is partially because you don't have to install plugins, as you would with Eclipse.

Pro: Great For Web Development: NetBeans offers support for HTML5, among other web technologies. So, if your team is focused on web development, NetBeans could be a good option.

Eclipse

A Java-based open source platform that enables the creation of highly customized IDEs from plug-in components built by Eclipse members. The platform is user-friendly for beginners and also suitable for the creation of more sophisticated applications. Eclipse includes a lot of plug-ins that allow developers to develop and test code written in other languages.

Pro: Tons of Plugins: Eclipse supports tons of plugins. Users can even make their own Plugin Development Environment (PDE).

The plugins for Eclipse are various. And many of the products we develop at Perforce have Eclipse plugins.

[GO to index page](#)

For example, Perforce version control software — Helix Core — has a plugin for Eclipse developers. It's called Helix Plugin for Eclipse (P4Eclipse). If you're already using Helix Core and Eclipse, adding this plugin will improve your workflows.

Pro: Supports Modelling: Eclipse also provides modelling support. This includes UML projects.

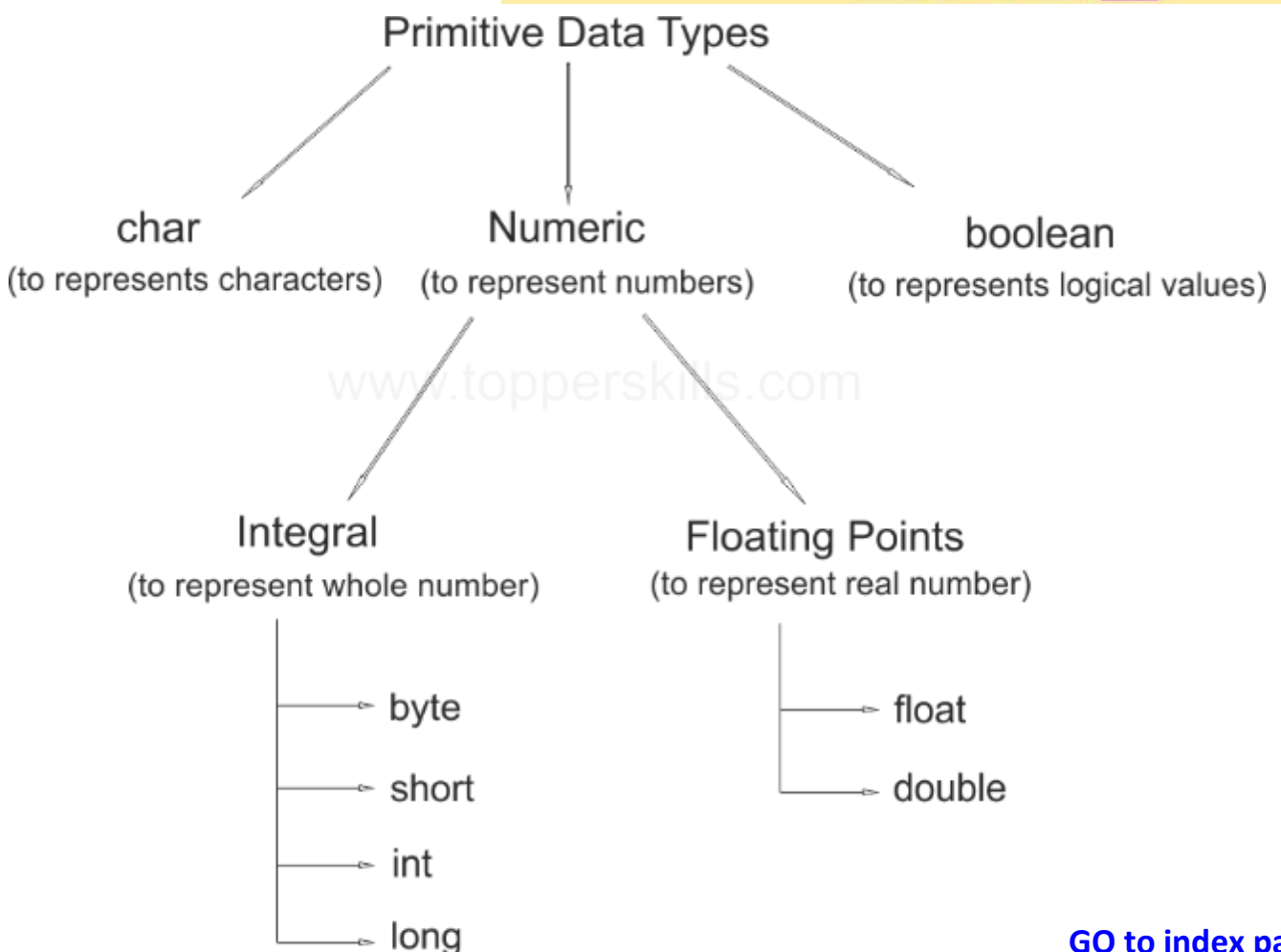
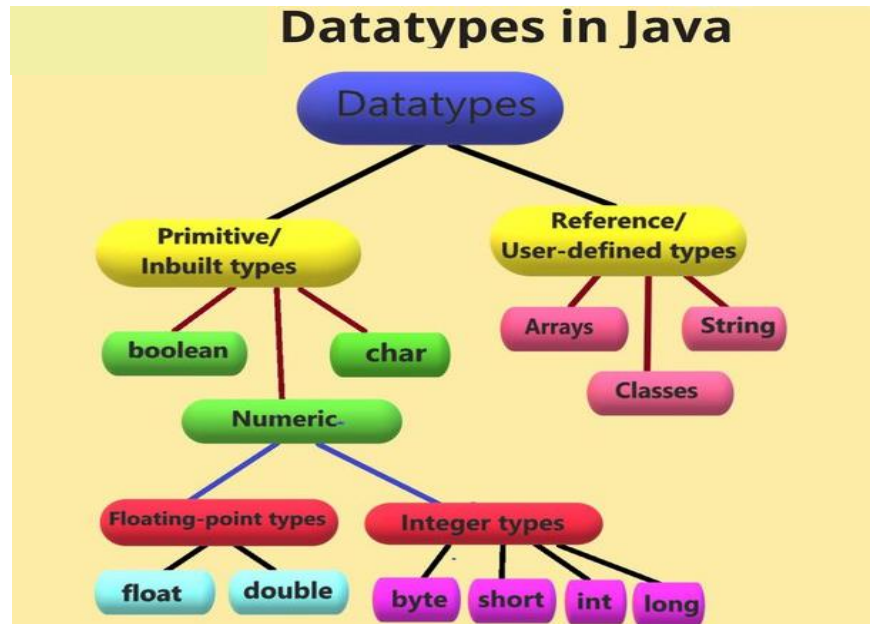
Pro: Great For Larger Projects: Eclipse is designed to handle large development projects. So, if your team works on large development projects, it may be wise to use Eclipse.

Data Type (Integer, Float, Character, Boolean)

Data Types in Java

Java language has a rich implementation of data types. Data types specify size and the type of values that can be stored in an identifier. In java, data types are classified into two categories:

1. **Primitive Data type**
2. **Non-Primitive Data type**



1) Primitive Data type

A primitive data type can be of eight types:

Primitive Data types							
char	boolean	byte	Short	int	long	float	double

Once a primitive data type has been declared its type can never change, although in most cases its value can change. These eight primitive type can be put into four groups

Integer

This group includes byte, short, int, long

byte : It is 8 bit integer data type. Value range from -128 to 127. Default value zero. example: byte b=10;

short : It is 16 bit integer data type. Value range from -32768 to 32767. Default value zero. example: short s=11;

int : It is 32 bit integer data type. Value range from -2147483648 to 2147483647. Default value zero. example: int i=10;

long : It is 64 bit integer data type. Value range from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807. Default value zero. example: long l=100012;

Floating-Point Number

This group includes float, double

float : It is 32 bit float data type. Default value 0.0f. example: float ff=10.3f;

double : It is 64 bit float data type. Default value 0.0d. example: double db=11.123;

Characters

This group represents char, which represent symbols in a character set, like letters and numbers.

char : It is 16 bit unsigned Unicode character. Range 0 to 65,535. example: char c='a';

Boolean

This group represent boolean, which is a special type for representing true/false values. They are defined constant of the language. example: **boolean** b=true;

Java Tokens

In Java, the program contains classes and methods. Further, the methods contain the expressions and statements required to perform a specific operation.

These statements and expressions are made up of **tokens**.

In other words, we can say that the expression and statement is a set of **tokens**. The tokens are the small building blocks of a Java program that are meaningful to the Java compiler.

The Java compiler breaks the line of code into text (words) is called **Java tokens**. These are the smallest element of the Java program. The Java compiler identified these words as tokens.

These tokens are separated by the delimiters. It is useful for compilers to detect errors. Remember that the delimiters are not part of the Java tokens.

[GO to index page](#)



Types of Tokens

- [Keywords](#)
- [Identifiers](#)
- [Literals](#)
- [Operators](#)
- [Separators](#)
- [Comments](#)
- [Whitespace](#)

Keywords: These are the **pre-defined** reserved words of any programming language. Each keyword has a special meaning. It is always written in lower case. Java provides the following keywords:

01. abstract	02. boolean	03. byte	04. break	05. class
06. case	07. catch	08. char	09. continue	10. default
11. do	12. double	13. else	14. extends	15. final
16. finally	17. float	18. for	19. if	20. implements
21. import	22. instanceof	23. int	24. interface	25. long
26. native	27. new	28. package	29. private	30. protected
31. public	32. return	33. short	34. static	35. super
36. switch	37. synchronized	38. this	39. thro	40. throws
41. transient	42. try	43. void	44. volatile	45. while
46. assert	47. const	48. enum	49. goto	50. strictfp

Identifier: Identifiers are used to name a variable, constant, function, class, and array. It usually defined by the user. The label is also known as a special kind of identifier that is used in the goto statement. There are some rules to declare identifiers are:

- **Identifiers are case sensitive.**
- **Identifiers must begin with a letter, dollar sign or underscore.**
- **Apart from the first character, an identifier can have any combination of characters.**
- **Identifiers in Java are case sensitive.**
- **Java Identifiers can be of any length.**
- **Identifier name cannot contain white spaces.**
- **Any identifier name must not begin with a digit but can contain digits within.**
- **Most importantly, keywords can't be used as identifiers in Java.**

Literals: In programming literal is a notation that represents a fixed value (constant) in the source code. It can be categorized as an **integer** literal, **string** literal, **Boolean** literal, etc. It is defined by the programmer. Once it has been defined cannot be changed. Java provides five types of literals are as follows:

- Integer
- Floating Point
- Character
- String
- Boolean

Literal	Type
23	int
9.86	double
false, true	boolean
'K', '7', '-'	char
"javatpoint"	String
null	any reference type

Operators: In programming, operators are the special symbol that tells the compiler to perform a special operation. Java provides different types of operators that can be classified according to the functionality they provide. There are eight types of operators in Java, are as follows:

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Unary Operators
- Logical Operators
- Ternary Operators
- Bitwise Operators
- Shift Operators

Operator	Symbols
Arithmetic	+, -, /, *, %
Unary	++, --, !
Assignment	=, +=, -=, *=, /=, %=, ^=
Relational	==, !=, <, >, <=, >=
Logical	&&,
Ternary	(Condition)? (Stat1) : (Stat2);
Bitwise	&, , ^, ~
Shift	<<, >>, >>>

Separators: The separators in Java is also known as **punctuators**. There are nine separators in Java, are as follows:

separator <= ; , | . () { } []

Note that the first three separators (;, and .) are tokens that separate other tokens, and the last six (3 pairs of braces) separators are also known as delimiters.

For example, Math.pow(9, 3);

[GO to index page](#)

- **Square Brackets []:** It is used to define array elements. A pair of square brackets represents single-dimensional array, two pairs of square brackets represent the two-dimensional array.
- **Parentheses ():** It is used to call the functions and parsing the parameters.
- **Curly Braces {}:** The curly braces denote the starting and ending of a code block.
- **Comma (,):** It is used to separate two values, statements, and parameters.
- **Assignment Operator (=):** It is used to assign a variable and constant.
- **Semicolon (;):** It is the symbol that can be found at end of the statements. It separates the two statements.
- **Period (.):** It separates the package name from the sub-packages and class. It also separates a variable or method from a reference variable.

Comments: Comments allow us to specify information about the program inside our Java code. Java compiler recognizes these comments as tokens but excludes it from further processing. The Java compiler treats comments as whitespaces. Java provides the following two types of comments:

- **Line Oriented:** It begins with a pair of forwarding slashes (//).
- **Block-Oriented:** It begins with /* and continues until it finds */.

The JDK offers developers a choice of three Java comment types:

1. Inline Java comments, which begin with two slashes //
2. Block Java comments, which follow the structure /* */
3. JavaDoc comments on methods and classes /** */

Whitespace: White space is defined as the ASCII space character, horizontal tab character, form feed character, and line terminator characters.

- Blank space, New line, Carriage return, Horizontal tab

Operators

Arithmetic Operators

Arithmetic operators are used to perform arithmetic operations on variables and data. For example, `a + b`. Here, the `+` operator is used to add two variables `a` & `b`. Similarly, there are various other arithmetic operators in Java.

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo Operation (Remainder after division)

Assignment Operators

Operator	Example	Equivalent to
=	<code>a = b;</code>	<code>a = b;</code>
+=	<code>a += b;</code>	<code>a = a + b;</code>
-=	<code>a -= b;</code>	<code>a = a - b;</code>
*=	<code>a *= b;</code>	<code>a = a * b;</code>
/=	<code>a /= b;</code>	<code>a = a / b;</code>
%=	<code>a %= b;</code>	<code>a = a % b;</code>

Assignment operators are used in Java to assign values to variables. For example, `int age; age = 5;`. Here, `=` is the assignment operator. It assigns the value on its right to the variable on its left. That is, 5 is assigned to the variable `age`. Let's see some more short hand assignment operators available in Java.

[GO to index page](#)

Relational Operators

Relational operators are used to check the relationship between two operands. For example, $a < b$; /*check if a is less than b*/ → Here, $<$ is the relational operator. It checks if a is less than b or not. It returns either true or false.

Operator	Description	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> returns false
<code>!=</code>	Not Equal To	<code>3 != 5</code> returns true
<code>></code>	Greater Than	<code>3 > 5</code> returns false
<code><</code>	Less Than	<code>3 < 5</code> returns true
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> returns false
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> returns true
<code><<=</code>	Left shift AND assignment operator	<code>C <<= 2</code> is same as <code>C = C << 2</code>
<code>>>=</code>	Right shift AND assignment operator	<code>C >>= 2</code> is same as <code>C = C >> 2</code>
<code>&=</code>	Bitwise AND assignment operator	<code>C &= 2</code> is same as <code>C = C & 2</code>
<code>^=</code>	bitwise exclusive OR and assignment operator	<code>C ^= 2</code> is same as <code>C = C ^ 2</code>
<code> =</code>	bitwise inclusive OR and assignment operator	<code>C = 2</code> is same as <code>C = C 2</code>

Unary Operators

Unary operators are used with only one operand. For example, `++` is a unary operator that increases the value of a variable by 1. That is, `++5` will return 6.

Operator	Meaning
<code>+</code>	Unary plus: not necessary to use since numbers are positive without using it
<code>-</code>	Unary minus: inverts the sign of an expression
<code>++</code>	Increment operator: increments value by 1
<code>--</code>	Decrement operator: decrements value by 1
<code>!</code>	Logical complement operator: inverts the value of a boolean

Bitwise Operators

Bitwise operators in Java are used to perform operations on individual bits. For example, Bitwise complement Operation of 35

`35 = 00100011` (In Binary)

`~ 00100011`

`11011100 = 220` (In decimal)

Here, `~` is a bitwise operator. It inverts the value of each bit (0 to 1 and 1 to 0).

Operator	Description
<code>~</code>	Bitwise Complement
<code>&</code>	Bitwise AND
<code> </code>	Bitwise OR
<code>^</code>	Bitwise exclusive OR
<code>>></code>	Right Shift
<code><<</code>	Left Shift
<code>>>></code>	Unsigned Right Shift

Ternary Operator

The ternary operator (conditional operator) is shorthand for the if-then-else statement.

For example, `variable = Expression ? expression1 : expression2`

Here's how it works.

If the Expression is true, expression1 is assigned to the variable.

If the Expression is false, expression2 is assigned to the variable.

[GO to index page](#)

Logical Operators

Logical operators are used to check whether an expression is true or false. They are used in decision making.

Operator	Example	Meaning
&& (Logical AND)	expression1 && expression2	true only if both expression are true
(Logical OR)	expression1 expression2	true if any one expression is true
! (Logical NOT)	!expression	true if expression is false and vice versa

Java Keywords

ASSERT

The assert keyword in Java is used for debugging purposes.

It allows developers to test assumptions about their code. Assertions can be enabled or disabled when the program is run, making them a flexible tool for catching and diagnosing errors during development.

The assert keyword is used to create an assertion, which is a statement that you believe to be true at a particular point in your code. If the assertion is false, the Java runtime will throw an Assertion Error.

Assertions are typically used to validate internal invariants and assumptions.

By default assertion are disabled in JVM. To enable it, we have to pass VM arguments.

There are two options, first **-enableassertions**, or its short form **-ea**, enables assertions, and other **-disableassertions**, or its short form **-da**, disables assertions.

ENUM

An enumerated type defines a finite set of symbolic names and their values.

These symbolic names are usually called **enum** constants or named constants.

One way to define such constants is to declare them as final, static variables in a class (or interface).

Enums in Java are type-safe and has their own name-space.

It means your enum will have a type for example "MachineState" in below example and you can not assign any value other than specified in Enum Constants.

Enum constants are implicitly static and final and can not be changed once created. Since constants defined inside Enum in Java are final you can safely compare them using "==" equality operator.

Enum in java can be used as an argument on switch statement and with "case:" like int or char primitive type. This feature of java enum makes them very useful for switch operations.

You can not create instance of enums by using new operator in Java because constructor of Enum in Java can only be private and Enums constants can only be created inside Enums itself.

Enum in Java can implement the interface and override any method like normal class.

You can define abstract methods inside Enum in Java and can also provide different implementation for different instances of enum in java.

[GO to index page](#)

STRICTFP

The strictfp keyword is used to force the precision of floating point calculations (float or double) in Java conform to IEEE's 754 standard, explicitly. Without using strictfp keyword, the floating point precision depends on target platform's hardware, i.e. CPU's floating point processing capability.

In other words, using strictfp ensures result of floating point computations is always same on all platforms.

The strictfp keyword can be applied for classes, interfaces and methods.

Rules

strictfp cannot be applied for constructors.

If an interface or class is declared with strictfp, then all methods and nested types within that interface or class are implicitly strictfp.

strictfp cannot be applied for interface methods.

Java Type Casting

Java supports two types of castings –

1. **primitive data type casting and**
2. **reference type casting.**

- ✓ primitive data type casting is when you assign a value of one primitive data type to another type.
- ✓ Reference type casting is nothing but assigning one Java object to another object. It comes with very strict rules and is explained clearly in Object Casting.

Java data type casting comes with 2 flavours.

1. **Implicit casting**
2. **Explicit casting**

In Java, there are also known as:

- **Widening Casting** (**automatically**) - converting a smaller type to a larger type size
byte -> short -> char -> int -> long -> float -> double
Note: This is also known as **Implicit Type Casting**.
- **Narrowing Casting** (**manually**) - converting a larger type to a smaller size type
double -> float -> long -> int -> char -> short -> byte
Note: This is also known as **Explicit Type Casting**.

Decision making statement

Selection statements are also known as branching statements or conditional or decision-making statements.

Here are two types of decision making statements in Java. They are:

1. **IF statements**
2. **switch statements**

[GO to index page](#)

(1) IF Statement:

(a) if...Statement:

IF statement consists of a Boolean expression followed by one or more statements.

Syntax:

```
if(Boolean_expression)
{
    //Statements will execute if the Boolean expression is true
}
```

If the Boolean expression evaluates to true then the block of code inside the if statement will be executed. If not the first set of code after the end of the if statement (after the closing curly brace) will be executed.

Example:

```
public class Test {
    public static void main(String args[]) {
        int x = 10;
        if( x < 20 )
        {
            System.out.print("This is if statement");
        }
    }
}
```

This would produce the following result:

This is if statement

(b) if...else Statement:

A if statement can be followed by an optional else statement, which executes when the Boolean expression is false.

Syntax:

```
if(Boolean_expression)
{
    //Executes when the Boolean expression is true
}
else
{
    //Executes when the Boolean expression is false
}
```

Example:

```
public class Test {
    public static void main(String args[]) {
        int x = 30;
        if( x < 20 )
        {
            System.out.print("This is if statement");
        }
        else
        {
            System.out.print("This is else statement");
        }
    }
}
```

This would produce the following result:

This is else statement

(c) The if...else if...else Statement: (Multiple IF)

if statement can be followed by an optional else if...else statement, which is very useful to test various conditions using single if...else if statement.

When using if , else if , else statements there are few points to keep in mind.

- if can have zero or one else's and it must come after any else if's.
- if can have zero to many else if's and they must come before the else.
- Once an else if succeeds, none of the remaining else if's or else's will be tested.

Syntax:

```
if(Boolean_expression 1)
{
    //Executes when the Boolean expression 1 is true
}
else if(Boolean_expression 2)
{
    //Executes when the Boolean expression 2 is true
}
else if(Boolean_expression 3)
{
    //Executes when the Boolean expression 3 is true
}
.
.
.
.
else
{
    //Executes when the none of the above condition is true.
}
```

(d) Nested if...else Statement:

It is always legal to nest if-else statements which means you can use one if or else if statement inside another if or else if statement.

Syntax:

```
if(Boolean_expression 1)
{
    //Executes when the Boolean expression 1 is true
    if(Boolean_expression 2)
    {
        //Executes when the Boolean expression 2 is true
    }
}
```

You can nest else if...else in the similar way as we have nested if statement.

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int x = 30, y = 10;  
        if( x == 30 )  
        {    if ( y == 10 ) { System.out.print("X = 30 and Y = 10"); }    }  
    }  
}
```

This would produce the following result:

X = 30 and Y = 10

(2) The switch Statement:

A switch statement allows a variable to be tested for equality against a list of values. Each value is called a case, and the variable being switched on is checked for each case.

Syntax:

```
switch(expression)  
{  
    case value :  
        //Statements  
        break; //optional  
    case value :  
        //Statements  
        break; //optional  
    .  
    .  
    .  
    .  
    //You can have any number of case statements.  
    default : //Optional  
        //Statements  
}
```

The following rules apply to a switch statement:

- The variable used in a switch statement can only be a **byte, short, int, or char.**
- You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
- The value for a case must be the same data type as the variable in the switch and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached.
- When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.
- A switch statement can have an optional default case. The default case can be used for performing a task when none of the cases is true.

[GO to index page](#)

Example:

```
public class Test {  
    public static void main(String args[]) {  
        char grade = 'C';  
        switch(grade)  
        {  
            case 'D' :  
                System.out.println("You passed");  
            case 'F' :  
                System.out.println("Better try again");  
                break;  
            case 'A' :  
                System.out.println("Excellent!");  
                break;  
            case 'C' :  
            case 'B' :  
                System.out.println("Well done");  
                break;  
            default :  
                System.out.println("Invalid grade");  
        }  
        System.out.println("Your grade is " + grade);  
    }  
}
```

This would produce the following result:

Well done
Your grade is a C

Loops in Java

There may be a situation when we need to execute a block of code several number of times, and is often referred to as a loop.

Java has very flexible three looping mechanisms. You can use one of the following three loops:

- (1) **while Loop**
- (2) **do...while Loop**
- (3) **for Loop** (As of Java 5, the enhanced for loop (**for each**) was introduced. This is mainly used for Arrays.)

(1) while Loop:

A while loop is a control structure that allows you to repeat a task a certain number of times.

Syntax:

```
while(Boolean expression)  
{  
    //Statements  
}
```

[GO to index page](#)

When executing, if the boolean_expression result is true, then the actions inside the loop will be executed. This will continue as long as the expression result is true.

Here, key point of the while loop is that the loop might not ever run. When the expression is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int x = 10;  
        while( x < 12 )  
        {  
            System.out.print("\n value of x : " + x++ );  
        }  
    }  
}
```

This would produce the following result:

value of x : 10

value of x : 11

(2) do...while Loop:

A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.

Syntax:

```
do  
{  
    //Statements  
} while(Boolean expression);
```

Notice that the Boolean expression appears at the end of the loop, so the statements in the loop execute once before the Boolean is tested.

If the Boolean expression is true, the flow of control jumps back up to do, and the statements in the loop execute again. This process repeats until the Boolean expression is false.

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int x = 10;  
        do  
        {  
            System.out.print("\n value of x : " + x++ );  
        }while( x <= 12 );  
    }  
}
```

This would produce the following result:

value of x : 10

value of x : 11

value of x : 12

(3) for Loop:

A for loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times. A for loop is useful when you know how many times a task is to be repeated.

[GO to index page](#)

Syntax:

```
for(initialization; Boolean_expression; increment/decrement)
{
    //Statements
}
```

Here is the flow of control in a for loop:

- The initialization step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the Boolean expression is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement past the for loop.
- After the body of the for loop executes, the flow of control jumps back up to the update statement. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the Boolean expression.
- The Boolean expression is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then update step, then Boolean expression). After the Boolean expression is false, the for loop terminates.

Example:

```
public class Test {
    public static void main(String args[]) {
        for(int x = 10; x < 20; x = x+3)
        {
            System.out.print("\n value of x : " + x );
        }
    }
}
```

This would produce the following result:

```
value of x : 10
value of x : 13
value of x : 16
value of x : 19
```

(4) Enhanced for loop in Java:

As of Java 5, the enhanced for loop was introduced. This is mainly used for Arrays.

Syntax:

```
for(declaration : expression)
{
    //Statements
}
```

- **Declaration:** The newly declared block variable, which is of a type compatible with the elements of the array you are accessing. The variable will be available within the for block and its value would be the same as the current array element.
- **Expression:** This evaluates to the array you need to loop through. The expression can be an array variable or method call that returns an array.

[GO to index page](#)

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int [] numbers = {10, 20, 30, 40, 50};  
        for(int x : numbers )  
        {  
            System.out.print( x + ", " );  
        }  
        System.out.print("\n");  
        String [] names ={"OM", "Jay", "Vijay", "Ajay"};  
        for( String name : names )  
        {  
            System.out.print( name + ", " );  
        }  
    }  
}
```

This would produce the following result:

10, 20, 30, 40, 50,
OM, Jay, Vijay, Ajay,

Jumping Statements

The break Keyword:

The break keyword is used to stop the entire loop. The break keyword must be used inside any loop or a switch statement. The break keyword will stop the execution of the innermost loop and start executing the next line of code after the block.

Syntax: The syntax of a break is a single statement inside any loop:

`break;`

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int [] numbers = {10, 20, 30, 40, 50};  
        for(int x : numbers )  
        {  
            if( x == 30 ) { break; }  
            System.out.print("\n" + x );  
        }  
    }  
}
```

This would produce the following result:

10
20

The continue Keyword:

The continue keyword can be used in any of the loop control structures. It causes the loop to immediately jump to the next iteration of the loop. In a for loop, the **continue keyword** causes flow of control to immediately jump to the update statement.

In a while loop or do/while loop, flow of control immediately jumps to the Boolean expression.

[GO to index page](#)

Syntax: The syntax of a continue is a single statement inside any loop:

`continue;`

Example:

```
public class Test {  
    public static void main(String args[]) {  
        int [] numbers = {10, 20, 30, 40};  
        for(int x : numbers )  
        {  
            if( x == 30 ) { continue; }  
            System.out.print("\n" + x );  
        }  
    }  
}
```

This would produce the following result:

```
10  
20  
40
```

The return Keyword:

In Java, the **return** keyword returns a value from a method. The method will return the value immediately when the keyword is encountered. This means that the method will not execute any more statements beyond the **return** keyword, and any local variables created in the method will be discarded.

This is similar to how a break statement immediately exits a for loop and transfers control back to the enclosing code block. To understand how this works, consider the following **example:**

```
public String fun(int a)  
{  
    if(a<10)  
        return "Number less than 10";  
    if(a<100)  
        return "Number greater than 10 but less than 100";  
    if(a>=100)  
        return "Number greater than or equal to 100";  
}
```

In the code above, if you pass the number 8 to the function, it will return the string “Number less than 10” and move on to the **function’s caller**.

Similarly, if parameter a is less than 100, the function returns “Number greater than 10 but less than 100.” If the parameter is greater than or equal to 100, the program returns “Number greater than or equal to 100.”

Every Java method must include a return type in its declaration. If there’s nothing to return, use **void** as the return type.

There are several return types classified in one of two ways.

Primitive types like **int**, **float**, and double represent raw values, whereas non-primitive types can represent things like classes, arrays, or interfaces.

[**GO to index page**](#)

Non-Primitive (Reference) Data type

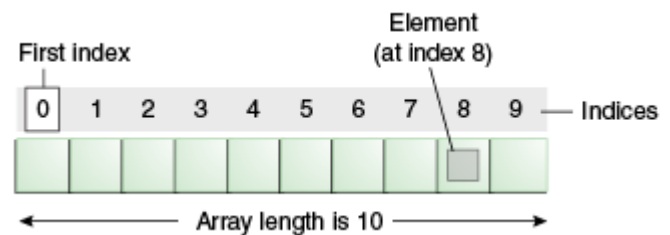
A reference data type is used to refer to an object. A reference variable is declared to be of specific type and that type can never change. Classes, Arrays, Interface are an example of **Non Primitive types**.

Java Array

Normally, array is a collection of similar type of elements that have contiguous memory location.

Java array is an object that contains elements of similar data type. It is a data structure where we store similar elements. We can store only fixed set of elements in a java array.

Array in java is index based, first element of the array is stored at 0 index.



Advantage of Java Array

- **Code Optimization:** It makes the code optimized, we can retrieve or sort the data easily.
- **Random access:** We can get any data located at any index position.

Disadvantage of Java Array

- **Size Limit:** We can store only fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in java.

Types of Array in java

There are two types of array.

1. Single Dimensional Array

2. Multidimensional Array

1. Single Dimensional Array in java

It is a simple array which can store the multiple elements inside which must belong to the same data type.

Syntax : `dataType[] arr;` (or) `dataType arr[];`

2. Multidimensional array in java

In such case, data is stored in row and column based index (also known as matrix form). There are three types of array.

1. Multidimensional Array
2. Rectangular Array
3. Jagged Array

Multidimensional Array

Syntax : `dataType[][] arrayRefVar;` (or) `dataType [][]arrayRefVar;` (or)
`dataType arrayRefVar[][];` (or) `dataType []arrayRefVar[];`

Example to instantiate Multidimensional Array in java

```
int[][] arr=new int[3][3]; //3 row and 3 column
```

Example to initialize Multidimensional Array in java

```
arr[0][0]=1;     arr[0][1]=2;     arr[0][2]=3;     arr[1][0]=4;     arr[1][1]=5;  
arr[1][2]=6;     arr[2][0]=7;     arr[2][1]=8;     arr[2][2]=9;     GO to index page
```

Rectangular array in java

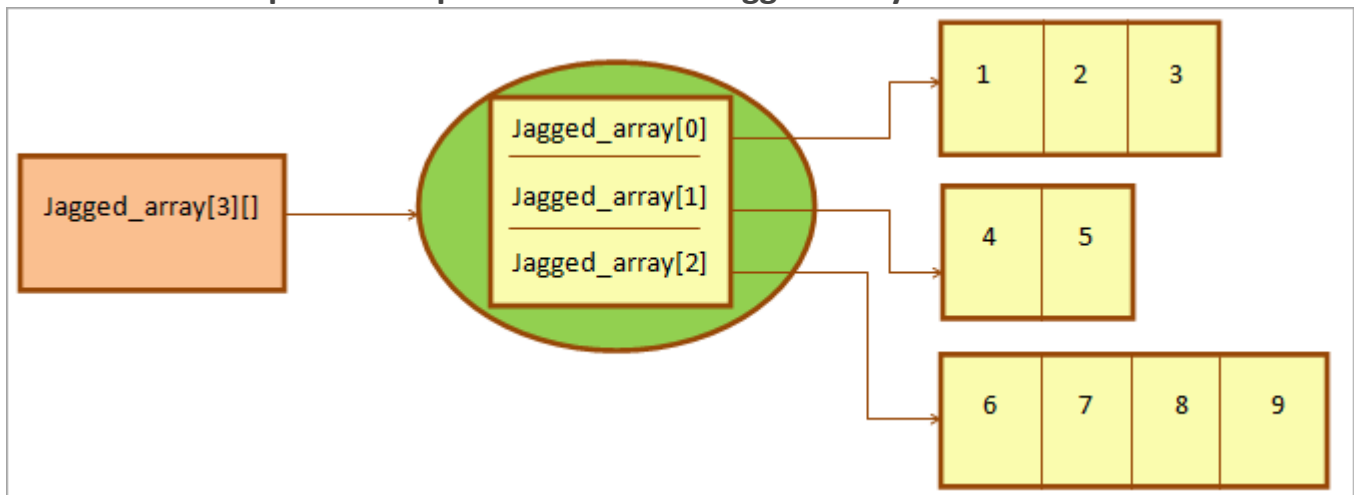
Rectangular arrays are, as the name implies, arrays which look like a table. A 2D rectangular array can be represented as a plane rectangle, and a 3D rectangular array can be represented as a cuboid. [4] Whatever the case, the idea is the same – in rectangular arrays, every row has the same number of columns.

Jagged array in java

It is an array of arrays where each element is, in turn, an array. A special feature of this type of array is that it is a Multidimensional array whose each element can have different sizes.

For Example, a two-dimensional array in Java is an array of single dimension array. In the case of a two-dimensional array, each one-dimensional array will have different columns.

Given below is a pictorial representation of a Jagged array.



From the above pictorial representation, we got an idea of how does it look. Above shown is a two-dimensional Jagged array. Each individual element of this array is a one-dimensional array that has varied sizes as shown above.

The first 1D array has 3 columns; the second row has 2 columns while the third has 4 columns. Create & Initialize Jagged Array

While creating an array of arrays you only specify the first dimension that represents a number of rows in the array.

You can create a two-dimensional jagged array as follows:

```
int myarray[][] = new int[3][];
```

In the above declaration, a two-dimensional array is declared with three rows.

Once the array is declared, you can define it as a Jagged array as shown below:

```
myarray[1] = new int[2];
```

```
myarray[2] = new int[3];
```

```
myarray[3] = new int[4];
```

The first statement above indicates that the first row in the 2D array will have 2 columns.

The second row will have 3 columns while the third row will have 4 columns thereby making it a Jagged array.

Once the array is created, you can initialize it with values. Note that if you don't explicitly initialize this array (as in the above case), then it will take the default values as initial values depending on the data type of the array.

[GO to index page](#)

Initializing a Jagged array is as shown below:

```
int[][]myarray={ new int[] { 1, 2, 3 }; new int[] { 4, 5, 6, 7 }; new int[] { 8, 9 }; };
```

As you can see above, the new operator is omitted and the array is initialized as well as declared in the same statement.

You can also omit all the new operators altogether and have a declaration and initialization statement as shown below.

```
int[][] arr = { { 1, 2, 3 }, { 4, 5, 6, 7 }, { 8, 9 } };
```

The program below initializes a ragged array by assigning initial values to each row. Here each row of the array is initialized to the column values.

Example:

```
class Main {
    public static void main(String[] args) {
        int myarray[][] = new int[3][]; // Declare a 2-D array with 3 rows
        // define and initialize jagged array
        myarray[0] = new int[]{1,2,3};
        myarray[1] = new int[]{4,5};
        myarray[2] = new int[]{6,7,8,9,10};
        // display the jagged array
        System.out.println("Two dimensional Jagged Array:");
        for (int i=0; i<myarray.length; i++) {
            for (int j=0; j<myarray[i].length; j++) System.out.print(myarray[i][j] + " ");
            System.out.println();
        }
    }
}
```

Command Line Argument Array

The java command-line argument is an argument i.e. passed at the time of running the java program.

The arguments passed from the console can be received in the java program and it can be used as an input.

So, it provides a convenient way to check the behavior of the program for the different values. You can pass N (1,2,3 and so on) numbers of arguments from the command prompt.

Simple example of command-line argument in java

In this example, we are receiving only one argument and printing it. To run this java program, you must pass at least one argument from the command prompt.

Example:

```
class CommandLineExample {
    public static void main(String args[]) {
        System.out.println("Your first argument is: "+args[0]);
    }
}
```

compile by > javac CommandLineExample.java

run by > java CommandLineExample OM

Output: Your first argument is: OM

[GO to index page](#)

OOP Concepts

(Class, Object, Encapsulation, Inheritance, Polymorphism)

Object Oriented Programming

In this page, we will learn about basics of OOPs. **Object Oriented Programming** is a paradigm that provides many concepts such as inheritance, data binding, polymorphism etc.

Simula is considered as the first object-oriented programming language. The programming paradigm where everything is represented as an object, is known as truly object-oriented programming language.

Smalltalk is considered as the first truly object-oriented programming language.

OOPs (**Object Oriented Programming System**)

Object means a real word entity such as pen, chair, table etc. Object-Oriented Programming is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:

1. **Object**
2. **Class**
3. **Inheritance**
4. **Polymorphism**
5. **Abstraction**
6. **Encapsulation**

Object

Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.

- ✓ object is a member (also called an instance) of a Java class.
- ✓ Object is a real world entity and runtime entity.
- ✓ objects are the things or the units of code that are eventually derived from the process.
- ✓ An object is an instance of a class.
- ✓ Objects have states and behaviors.
- ✓ An object is an entity that has a state, specific behavior, and has been created for a specific purpose.



Class

- ✓ Collection of objects is called class. It is a logical entity.
- ✓ It is also called a blue print of your whole programming structure which contains all the members of your class.
- ✓ class is a basic building block of an object-oriented language
- ✓ class is a logical template to create objects that share common properties and methods.
- ✓ class is Blueprint of an object which describes/ shows all the functions and data that are provided by an object of a specific class
- ✓ A class is a user defined blueprint or prototype from which objects are created.
- ✓ A class can be defined as a template/blueprint that describes the behavior/state that the object of its type support.

[GO to index page](#)

Difference between object and class

There are many differences between object and class. A list of differences between object and class are given below:

No.	Object	Class
1)	Object is an instance of a class.	Class is a blueprint or template from which objects are created.
2)	Object is a real world entity such as pen, laptop, mobile, bed, keyboard etc.	Class is a group of similar objects.
3)	Object is a physical entity.	Class is a logical entity.
4)	Object is created through new keyword e.g. <code>Abc o1=new Abc();</code>	Class is declared using class keyword e.g. <code>class Abc{}</code>
5)	Object is created many times as per requirement.	Class is declared once.
6)	It allocates memory when it is created.	It doesn't allocated memory when it is created.

Inheritance

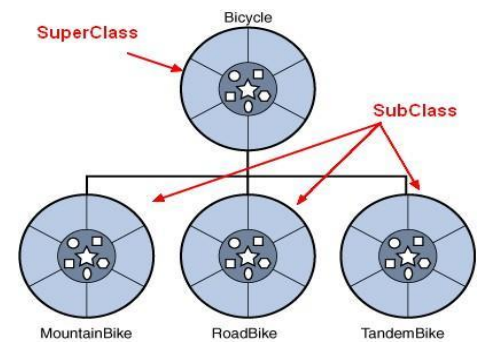
When one object acquires all the properties and behaviours of parent object i.e. known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Different Terms used in Inheritance

Parent Class - Child Class

Super Class - Sub Class

Base Class - Derived Class



Polymorphism

“Poly” means many and “Morph” means Forms. One Name Many Forms.

When one task is performed by different ways is known as polymorphism.

For example: to draw something e.g. shape or rectangle etc.

In java, we use method overloading and method overriding to achieve polymorphism.

Another example can be to speak something e.g. cat speaks meow, dog barks woof etc.



Abstraction

Hiding internal details and showing functionality is known as abstraction. For example: phone call, we don't know the internal processing. In java, we use abstract class and interface to achieve abstraction.

Encapsulation

Binding (or wrapping) code and data together into a single unit is known as encapsulation.

For example: capsule, it is wrapped with different medicines.

A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.



[GO to index page](#)

The following represents the data hiding (encapsulation) structure in class.

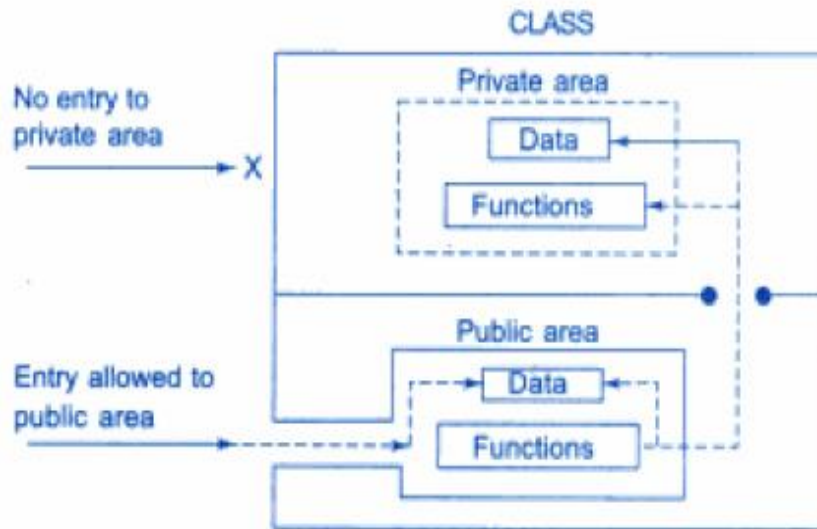
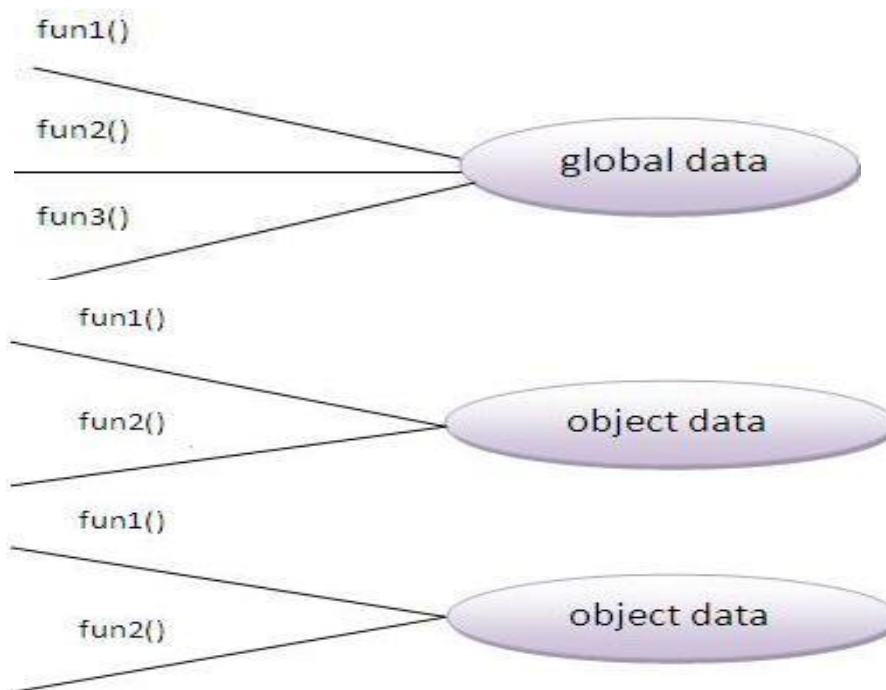


Fig. 5.1 ⇔ Data hiding in classes

Advantage of OOPs over Procedure-oriented programming language

- OOPs make development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grows.
- OOPs provide data hiding whereas in Procedure-oriented programming language a global data can be accessed from anywhere.
- OOPs provide ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language.

Difference between object-oriented programming language and object-based programming language



Object based programming language follows all the features of OOPs except Inheritance. JavaScript and VBScript are examples of object based programming languages.

[GO to index page](#)

Creating and using Class with members

- A class in Java is a set of objects which shares common characteristics/ behavior and common properties/ attributes.
- Class is not a real-world entity. It is just a template or blueprint or prototype from which objects are created.
- Class does not occupy memory.
- Class is a group of variables of different data types and a group of methods.

A Class in Java can contain:

Data member, Method, Constructor, Nested Class, Interface.

Example:

```
class Student {  
    int id;    // data member (Instance variable)  
    String name;    // data member (Instance variable)  
    public static void main(String args[]) {  
        Student s1 = new Student();// creating an object  
        System.out.println(s1.id);  
        System.out.println(s1.name);  
    }  
}
```

Output:

```
0  
null
```

Class Declaration in Java

```
access_modifier class <class_name>  
{  
    data member;  
    method;  
    constructor;  
    nested class;  
    interface;  
}
```

Constructor

- ✓ A constructor in Java is a special method that is used to initialize objects.
- ✓ In Java, Constructor is a block of codes similar to the method.
- ✓ It is called when an instance of the class is created.
- ✓ At the time of calling the constructor, memory for the object is allocated in the memory.
- ✓ It is a special type of method that is used to initialize the object.
- ✓ Every time an object is created using the new() keyword, at least one constructor is called.

It is not necessary to write a constructor for a class. It is because the java compiler creates a default constructor (constructor with no arguments) if your class doesn't have any.

How Java Constructors are Different From Java Methods?

- must have the same name as the class within which it is defined it is not necessary for the method in Java.
- Constructors do not return any type while method(s) have the return type or void if does not return any value.
- Constructors are called only once at the time of Object creation while method(s) can be called any number of times.

Types of Constructors in Java

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

[GO to index page](#)

1. Default Constructor (No-Argument Constructor) in Java

A default constructor is one that doesn't take any arguments. It's also called a no-argument constructor. If not provided, the Java compiler will automatically generate one for you.

2. Parameterized Constructor

A parameterized constructor takes arguments to initialize the object's attributes.

In addition, these types of constructors are usually used for constructor overloading to differentiate between multiple constructors with different data types.

3. Copy Constructor

A copy constructor is used to create an exact copy of an object. It takes an object of the same class as a parameter and copies its attributes.

finalize() method

The `java.lang.Object.finalize()` is called by the garbage collector on an object when garbage collection determines that there are no more references to the object. A subclass overrides the `finalize` method to dispose of system resources or to perform other cleanup.

There are other ways to release the used resources in Java like the `close()` method in case of file handling or `destroy()` method. But, the issue with these methods is they don't work automatically, we have to call them manually every time.

Declaration

Following is the declaration for `java.lang.Object.finalize()` method

Overriding is a feature in Java, where **parent class(superclass)** methods can be reimplemented in **child classes(subclass)**.

While **overriding** any parent class method: **name, return type, and parameters** of the method should be the **same in child class**.

Object class is the superclass of all the classes in Java hence, Object class methods like the `finalize()` method can be overridden.

Example

```
public class JavafinalizeExample1 {
    public static void main(String[] args) {
        JavafinalizeExample1 obj = new JavafinalizeExample1();
        System.out.println(obj.hashCode());
        obj = null;
        System.gc(); // calling garbage collector
        System.out.println("end of garbage collection"); }
    @Override
    protected void finalize()
    { System.out.println("finalize method called"); }
}
```

Output:

```
2018699554
end of garbage collection
finalize method called
```

Static and Non-Static Members

There are three types of variables in Java:

1. **Local Variables**
2. **Instance Variables**
3. **Static Variables**

The **Local** variables and **Instance** variables are together called **Non-Static variables**. Hence it can also be said that the Java variables can be divided into 2 categories:

- **Static Variables**:
 - When a variable is declared as **static**, then a single copy of the variable is created and shared among all objects at a class level. Static variables are, essentially, global variables.
 - All instances of the class share the same static variable.
 - Important points for static variables: We can create static variables at class level only.
- **Non-Static Variable**
 - Instance Variables**:
 - Instance variables are non-static variables and are declared in a class outside any method, constructor or block.
 - As instance variables are declared in a class, these variables are created when an object of the class is created and destroyed when the object is destroyed.
 - Unlike local variables, we may use access specifiers for instance variables. If we do not specify any access specifier then the default access specifier will be used.
 - Initialisation of Instance Variable is not Mandatory. Its default value is 0
 - Instance Variable can be accessed only by creating objects.

Local Variables:

- A variable defined within a block or method or constructor is called local variable.
- These variables are created when the block is entered or the function is called and destroyed after exiting from the block or when the call returns from the function.
- The scope of these variables exists only within the block in which the variable is declared. i.e. we can access this variable only within that block.
- Initialisation of Local Variable is Mandatory.
- **The main differences between static and non static variables are:**

Static variable	Non static variable
Static variables can be accessed using class name	Non static variables can be accessed using instance of a class
Static variables can be accessed by static and non-static methods	Non static variables cannot be accessed inside a static method.
Static variables reduce the amount of memory used by a program.	Non static variables do not reduce the amount of memory used by a program
Static variables are shared among all instances of a class.	Non static variables are specific to that instance of a class.
Static variable is like a global variable and is available to all methods.	It is like a local variable and they can be accessed through only instance of a class.
It is declared with the "static" keyword.	No need to declare with any keyword.

[GO to index page](#)

It is a member of a class but is not a member of an instance of that class.	It belongs to a class and is both a member and an instance of that class.
Only static methods and data members from the same class or another class may be accessed by it.	It can access static and non-static data members and methods of the class.
Memory allocation happens only once.	When the method is called, memory is allocated.
No object needs to be created in order to access it.	To access it, you must create a class object.
It uses compile-time or early binding.	It uses dynamic or runtime binding.
It cannot be overridden.	It can be overridden.

Overloading (Constructor & Method)

Java supports Constructor Overloading in addition to overloading methods. In Java, overloaded constructor is called based on the parameters specified when a new is executed.

The constructor overloading can be defined as the concept of having more than one constructor with different parameters so that every constructor can perform a different task.

When do we need Constructor Overloading?

Sometimes there is a need of initializing an object in different ways. This can be done using constructor overloading.

What is the Difference between Method Overloading and Constructor Overloading in Java?

Constructor is used to initialize an object whereas a method is used to perform certain functions of a class.

Constructors are invoked implicitly when an object is created whereas methods are called explicitly by the user.

Constructors don't have a return type whereas methods have a return type of either a type of Object or void.

Constructor should have the same name as that of the class whereas method name should not be of the same name as that of class.

Varargs, IIB (Instance Initialization Block) in Java

Varargs

Let's suppose you are creating a Java method. However, you are not sure how many arguments your method is going to accept. To address this problem, Java 1.5 introduced varargs. Varargs is a short name for variable arguments. In Java, an argument of a method can accept arbitrary number of values. This argument that can accept variable number of values is called varargs.

The syntax for implementing varargs is as follows:

```
accessModifier methodName(datatype... arg)
{
    // method body
}
```

[GO to index page](#)

In order to define vararg, ... (three dots) is used in the formal parameter of a method.

A method that takes variable number of arguments is called a variable-arity method, or simply a varargs method.

Example using varargs:

```
class Test1 {  
    // A method that takes number of integer arguments.  
    static void fun(int... a) {  
        System.out.println("Number of arguments: " + a.length);  
        for (int i : a) System.out.print(i + " "); // using for each loop to display contents of a  
    }  
    public static void main(String args[]) {  
        // Calling the varargs method with different number of parameters  
        fun(100);           // one parameter  
        fun(1, 2, 3, 4);    // four parameters  
        fun();              // no parameter  
    }  
}
```

Output

```
Number of arguments: 1  
100  
Number of arguments: 4  
1 2 3 4  
Number of arguments: 0
```

IIB (Instance Initialization Block)

Instance Initializer block is used to initialize the instance data member. It runs each time when an object of the class is created.

The initialization of the instance variable can be done directly but there can be performed extra operations while initializing the instance variable in the instance initializer block

use of instance initializer block

while we can directly assign a value in instance data member

For **example**:

```
class Bike{ int speed=100; }
```

Why use instance initializer block?

Suppose I have to perform some operations while assigning value to instance data member e.g. a for loop to fill a complex array or error handling etc.

```
class Bike {  
    int speed;  
    Bike7() {    System.out.println("speed is "+speed);    }  
    // instance initializer block  
    {    speed=100;    }  
}
```

[**GO to index page**](#)